

Universidade de Vigo

Deseño e desenvolvemento dunha funcionalidade de
busca e filtrado de cursos no catálogo de cursos en liña

Aarón Riveiro Vilar

Traballo Fin de Grao
Escola de Enxeñaría de Telecomunicación
Grao en Enxeñaría de Tecnoloxías de Telecomunicación

Titores:
Manuel Caeiro Rodríguez
Óscar Vázquez Trigo

Curso 2025/2026

Índice

1. Introducción	1
1.1. Contexto do proxecto	1
1.2. Motivación	1
1.3. A plataforma de partida	2
2. Obxectivos e requisitos	3
2.1. Requisitos funcionais	4
2.1.1. Ordenación de resultados	4
2.1.2. Busca difusa	4
2.1.3. Tradución automática da consulta	4
2.2. Requisitos non funcionais	5
3. Deseño	5
3.1. Ordenación de resultados	5
3.2. Busca difusa	7
3.3. Tradución automática da consulta	8
4. Desenvolvemento	9
4.1. Ordenación de resultados	9
4.2. Busca difusa	10
4.3. Tradución automática da consulta	11
5. Conclusións e liñas futuras	12
6. Referencias	12
7. Anexos	13
A. Estado da arte	13
A.1. Busca e indexación en sistemas de xestión de contidos	13
A.2. Busca difusa e correspondencia aproximada de cadeas	13
A.3. Selección do motor de tradución automática	14
B. Calibrado do algoritmo de busca difusa	14
B.1. Datos de proba	14
B.2. Batería de consultas	14
B.3. Parámetros e configuracións avaliadas	15
B.4. Resultados	15
B.5. Configuración seleccionada	16

1. Introducción

Nos últimos anos, o crecemento sostido dos sistemas de información dixitais deu lugar a repositorios con volumes de datos cada vez maiores e máis heteroxéneos. Neste contexto, a mera dispoñibilidade da información xa non resulta suficiente: é necesario dotar os sistemas de mecanismos eficaces que permitan localizar, filtrar e presentar os datos de forma relevante para o usuario. A disciplina da recuperación de información (*Information Retrieval*) estuda precisamente os métodos e modelos que permiten acceder de maneira eficiente a grandes coleccións de documentos, maximizando a relevancia dos resultados ofrecidos fronte a unha consulta determinada [1].

A medida que aumenta o tamaño e a complexidade dos repositorios, a experiencia de usuario pasa a depender en grande medida da calidade do sistema de busca e filtrado. Non só é importante a rapidez de resposta, senón tamén a precisión e exhaustividade dos resultados, así como a capacidade do sistema para interpretar a intención de busca do usuario e ofrecer mecanismos de refinamento progresivo mediante filtros, facetas ou criterios adicionais [2]. Un sistema de busca limitado a coincidencias literais sobre un único campo de texto pode resultar insuficiente en contornas nas que os datos presentan múltiples dimensións e atributos relevantes.

Por iso, o deseño e a implementación de estratexias avanzadas de busca e filtrado constitúen un elemento clave en calquera sistema de información orientado ao usuario final, especialmente cando se pretende facilitar o acceso eficiente a contidos estruturados e mellorar a usabilidade global da plataforma.

1.1. Contexto do proxecto

O proxecto DACEM (*Digitising Academic Catalogues for Enhanced Mobility*) [3], aprobado na convocatoria de 2023 do programa Erasmus+ para Asociacións Estratéxicas en Educación Superior (código de proxecto 2023-1-ES01-KA220-HED-000160344), ten como finalidade o desenvolvemento dunha plataforma de catálogo de cursos dirixida a institucións educativas europeas. O seu obxectivo principal é ofrecer un sistema que permita publicar e manter información actualizada sobre titulacións e materias de forma accesible, estruturada e interoperable, facilitando así os procesos de mobilidade académica internacional. A plataforma está orientada a dous perfís principais de usuarios: as institucións educativas, que a empregan para publicar e manter actualizado o seu catálogo de cursos; e os estudantes que, no marco de procesos de mobilidade Erasmus, consultan a oferta formativa das universidades parceiras.

O sistema de información asociado a esta plataforma está fundamentado en tecnoloxías de software libre e está concibido para poder despregarse tanto en contornas locais como en infraestruturas na nube. Máis alá do almacenamento estruturado de datos académicos, o correcto funcionamento do sistema depende da súa capacidade para xestionar, organizar e recuperar a información de maneira eficiente. Neste contexto, os mecanismos de busca e filtrado desempeñan un papel esencial, xa que constitúen o principal medio de interacción entre o usuario e o repositorio de cursos. A calidade destes mecanismos condiciona directamente a usabilidade do sistema, a precisión dos resultados obtidos e a experiencia global de navegación.

1.2. Motivación

Un dos retos fundamentais nos sistemas de información que xestionan grandes volumes de datos estruturados non reside unicamente no seu almacenamento, senón na forma en que estes datos son recuperados e presentados ao usuario. A medida que un repositorio medra en número de elementos e en complexidade de atributos, os mecanismos de busca simples baseados en coincidencias textuais directas poden resultar insuficientes para ofrecer resultados relevantes e axustados ás necesidades reais de consulta.

Esta problemática é especialmente significativa en contornas nas que os datos presentan múltiples dimensións descritivas (como titulacións, materias, áreas de coñecemento, idio-

mas, créditos ou modalidades) e nas que o usuario pode ter criterios de busca diversos e combinados. A ausencia de mecanismos avanzados de filtrado e refinamento progresivo limita a capacidade do sistema para adaptarse a diferentes perfís de usuario e escenarios de consulta, reducindo a eficiencia na localización de información pertinente.

No ámbito actual do desenvolvemento de aplicacións web e sistemas de xestión de contidos, as tendencias apuntan cara á implementación de motores de busca máis sofisticados, capaces de indexar múltiples campos, ponderar resultados segundo a relevancia, incorporar criterios de filtrado dinámico, xestionar consultas en múltiples idiomas e mellorar a experiencia de usuario mediante interfaces intuitivas. Estas solucións non só optimizan o acceso á información, senón que contribúen a unha maior usabilidade e aproveitamento do sistema no seu conxunto.

O presente Traballo de Fin de Grao enmárcase neste contexto, propoñendo o deseño e desenvolvemento dunha mellora nos mecanismos de busca e filtrado do catálogo de cursos da plataforma DACEM.

1.3. A plataforma de partida

A plataforma DACEM está implementada sobre Drupal 10, un sistema de xestión de contidos (CMS) de código aberto amplamente empregado no ámbito académico e institucional [4]. Drupal 10 proporciona unha contorna flexible para a definición de tipos de contido estruturado, a xestión de usuarios e a publicación de información a través de interfaces web configurables.

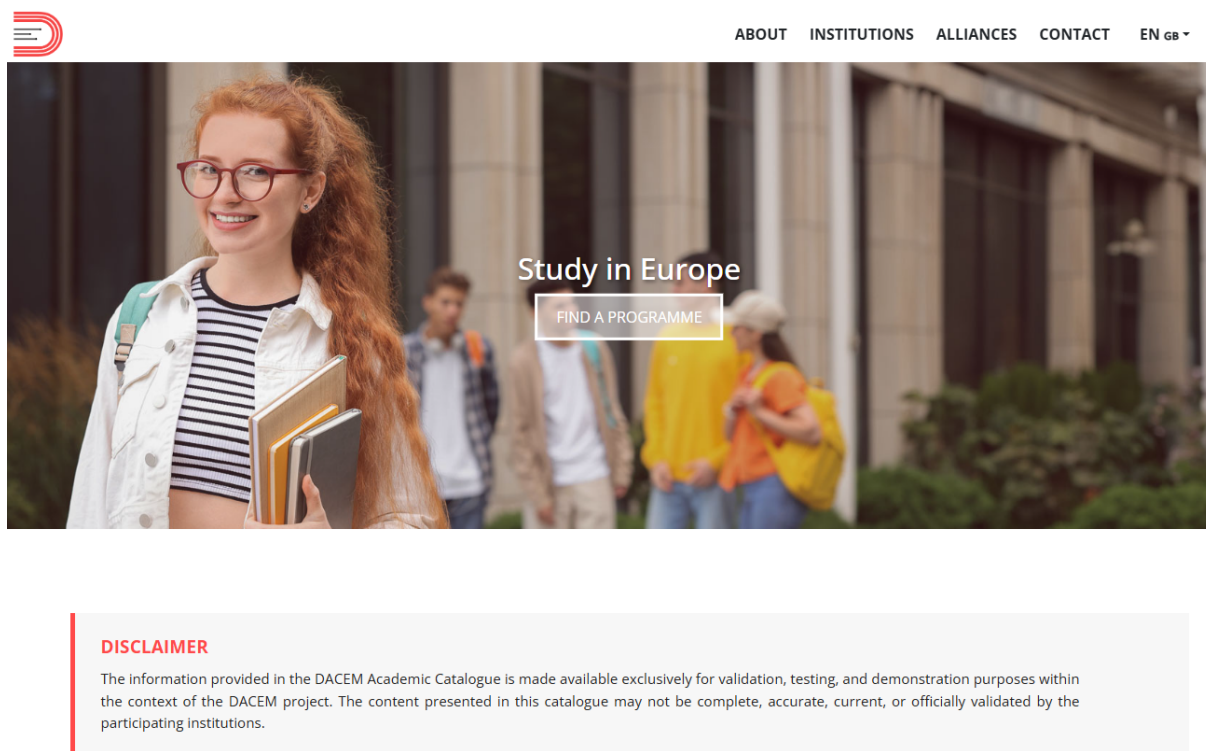


Figura 1: Páxina de inicio da plataforma DACEM.

O modelo de datos organízase arredor de tres tipos de contido principais: *programme*, que representa unha titulación académica; *individual educational component* (IEC), correspondente a unha materia ou compoñente curricular individual; e *institution*, que modela as universidades participantes. Cada entidade conta cun conxunto de campos estruturados que recollen atributos como a área de coñecemento, o nivel de cualificación, os créditos, o idioma de impartición ou o país de orixe, entre outros.

Para a presentación e consulta do catálogo, a plataforma emprega o módulo *Views*, integrado no núcleo de Drupal, que permite definir listaxes e consultas de contido de forma declarativa. O módulo *Better Exposed Filters* [5] complementa *Views* proporcionando a interface de filtrado ao usuario. Existen dúas vistas principais de busca: *search_programme*, accesible en `/search-programme`, orientada á consulta de titulacións; e *search_iec*, en `/search-iec`, orientada á consulta de compoñentes educativos individuais. Existe ademais a vista do catálogo de institución (*institution_new_catalogue*), que presenta os programas e compoñentes educativos ofertados por unha institución concreta, accesible en `/catalogue/{nid}/programmes` e `/catalogue/{nid}/iecs` respectivamente. A capa de presentación visual está baseada nun tema personalizado construído sobre Bootstrap 5.

O mecanismo de busca textual actual baséase no filtro *combine* de *Views*, que realiza comparacións de subcadeas sobre o campo título das entidades. Esta aproximación presenta limitacións relevantes:

- A busca restrinxese ao campo título, ignorando atributos como a descrición ou a área de coñecemento.
- Non se aplica ningunha ponderación por relevancia, polo que os resultados se presentan en orde alfabética con independencia da pertinencia da consulta.
- Non existe tolerancia a erros tipográficos, de xeito que calquera inexactitude na consulta pode producir cero resultados aínda que o elemento buscado exista no catálogo.
- Aínda que os contidos están dispoñibles en inglés e opcionalmente noutros idiomas, a busca non contempla a posibilidade de formular a consulta nun idioma distinto ao dos contidos, o que impide localizar resultados mediante termos non ingleses cando non existe tradución indexada.
- As opcións de filtrado son estáticas e non se adaptan ao conxunto de resultados dispoñibles, polo que poden aparecer valores sen correspondencia con ningunha entrada existente.

2. Obxectivos e requisitos

O presente traballo ten como obxectivo principal mellorar a experiencia de busca e navegación do catálogo de cursos da plataforma DACEM. Partindo das limitacións identificadas no sistema actual (ausencia de criterios de ordenación, busca por coincidencia exacta, sen tolerancia a erros tipográficos e imposibilidade de consultar en idiomas distintos ao dos contidos), propónse o deseño e implementación de tres melloras concretas:

- A incorporación de criterios de ordenación de resultados nas vistas de busca e na vista do catálogo de institución.
- A implementación dun mecanismo de busca difusa que permita localizar programas e compoñentes educativos aínda en presenza de erros tipográficos na consulta.
- A incorporación dun servizo de tradución automática da consulta que permita obter resultados pertinentes aínda cando os contidos non dispoñan de tradución ao idioma do usuario.

A continuación defínense os requisitos funcionais e non funcionais que guían o desenvolvemento. Enténdese por requisito funcional aquel que describe as capacidades concretas que debe ofrecer o sistema, mentres que os requisitos non funcionais establecen condicións relativas ao seu comportamento, integración e mantenibilidade, sen especificar funcionalidades concretas.

2.1. Requisitos funcionais

2.1.1. Ordenación de resultados

1. O sistema debe permitir ao usuario ordenar os resultados das vistas de busca segundo diferentes criterios, adaptados ao tipo de entidade que se consulta.
2. Cada criterio de ordenación debe permitir alternar entre orde ascendente e descendente mediante a mesma acción de usuario.
3. Todas as vistas deben contemplar os seguintes criterios de ordenación comúns: orde alfabética por título, campo de estudo e número de créditos.
4. As vistas de busca (*search_programme* e *search_iec*) deben incluír adicionalmente o criterio de ordenación por país de orixe da institución.
5. A vista de programas (*search_programme*) e a páxina de programas do catálogo de institución (*institution_new_catalogue*) deben incluír adicionalmente o criterio de ordenación por nivel de cualificación EQF (*European Qualifications Framework*).
6. A vista de compoñentes educativos (*search_iec*) e a páxina de compoñentes educativos do catálogo de institución deben incluír adicionalmente os criterios de ordenación por programa asociado e cuadrimestre.
7. A interface de ordenación debe integrarse visualmente de forma coherente co deseño existente da plataforma.

2.1.2. Busca difusa

1. O sistema debe ser capaz de localizar resultados relevantes aínda cando a consulta introducida polo usuario conteña erros tipográficos ou grafías aproximadas.
2. A busca difusa debe aplicarse ás vistas de busca e á vista do catálogo de institución.
3. A busca debe operar sobre múltiples campos indexados, de xeito que a consulta poida coincidir co título da entidade, co nome da institución asociada ou, no caso dos compoñentes educativos, co título do programa ao que pertencen.
4. A busca difusa debe aplicar un criterio de similaridade entre a consulta e os campos indexados dos elementos do catálogo, de maneira que se poidan recuperar resultados con pequenas desviacións ortográficas respecto ao texto almacenado.
5. A normalización da consulta debe incluír a conversión a minúsculas e a eliminación de diacríticos, de xeito que maiúsculas e caracteres acentuados non afecten aos resultados da busca.
6. A busca debe operar sobre todos os idiomas indexados, de xeito que o usuario poida localizar resultados con independencia do idioma no que estea configurada a interface ou do idioma no que estea almacenado o título.
7. Os resultados deben presentarse ordenados por relevancia, de xeito que os elementos con maior correspondencia coa consulta aparezan en primeiro lugar.
8. O mecanismo de busca difusa debe integrarse no fluxo de consulta existente, de xeito que o usuario non perciba diferenzas na interface respecto ao comportamento anterior.
9. O sistema debe manter un comportamento consistente ante consultas baleiras ou moi curtas, evitando erros ou resultados inesperados.

2.1.3. Tradución automática da consulta

1. O sistema debe ser capaz de localizar resultados pertinentes cando a consulta está formulada nun idioma distinto ao dos contidos almacenados, sen requirir que o usuario reformule a busca en inglés.

2. A tradución debe realizarse ao inglés, lingua na que se almacena obrigatoriamente todo o contido do catálogo.
3. O sistema debe dispoñer dun mecanismo de tradución en dous niveis: un dicionario estático de termos académicos como primeira opción, e un servizo de tradución automática como alternativa cando o termo non estea recollido nel.
4. O termo orixinal da consulta debe incluírse sempre xunto á tradución como medida de robustez, de xeito que a busca funcione correctamente aínda en caso de fallo na tradución ou cando o usuario escriba directamente en inglés, sen afectar á dispoñibilidade do sistema.

2.2. Requisitos non funcionais

1. A solución debe integrarse coa plataforma existente sen alterar a estrutura fundamental do modelo de datos nin comprometer o funcionamento do resto de funcionalidades do sistema.
2. O desenvolvemento debe estar fundamentado en tecnoloxías de código aberto, mantendo coherencia cos principios e a arquitectura do proxecto DACEM.
3. O sistema debe garantir un comportamento estable e previsible ante entradas incorrectas ou consultas mal formadas, evitando erros visibles para o usuario final.
4. A interface debe cumprir criterios básicos de usabilidade, asegurando que os mecanismos de interacción resulten intuitivos e coherentes co deseño xeral da plataforma.
5. O sistema debe garantir tempos de resposta axeitados nas consultas, de maneira que as melloras introducidas non degraden de forma perceptible a experiencia de uso.
6. A solución debe minimizar o impacto nos recursos do servidor, evitando sobrecargas innecesarias e optimizando as consultas realizadas á base de datos.
7. O rendemento do sistema debe manterse estable ante un incremento razoable do volume de datos almacenados no catálogo, asegurando que a solución escale de forma adecuada.
8. A implementación debe estruturarse de forma modular e documentada, de xeito que cada mellora poida ser mantida ou ampliada de forma independente sen afectar ao resto do sistema.

3. Deseño

3.1. Ordenación de resultados

Para incorporar a funcionalidade de ordenación á vista de programas, á vista de compoñentes educativos e á vista do catálogo de institución, optouse por empregar o mecanismo de ordenación exposta do módulo *Views* de Drupal, en lugar de implementar unha solución baseada en manipulación directa de consultas ou en ordenación no lado do cliente. Esta decisión responde a tres razóns principais:

- O mecanismo intégrase de forma nativa co sistema *AJAX* de *Views*, o que evita ter que reimplementar a actualización parcial de resultados.
- Mantén a coherencia coa arquitectura da plataforma, que xa emprega *Views* de forma extensiva para a xestión e presentación de contido.
- Os criterios de ordenación quedan declarados na configuración da vista en lugar de estar codificados en PHP, o que facilita a súa modificación sen necesidade de modificar código.

O mecanismo funciona do seguinte xeito: cando se declara un criterio de ordenación como exposto na configuración dunha vista, *Views* xera automaticamente dous campos de control no formulario asociado, un que identifica o criterio polo que ordenar e outro que

indica a dirección (ascendente ou descendente), e reconstrúe a consulta ao recibir o envío do formulario.

Para a interacción do usuario, deseñouse un comportamento de alternancia: o primeiro clic sobre un botón de ordenación establece o criterio en orde ascendente; un segundo clic sobre o mesmo botón inverte a dirección de ordenación, como se mostra na figura 2. Esta decisión permite concentrar cada criterio nun único botón, evitando duplicar os controis en dúas versións por dirección. A lóxica de alternancia, implementada en *JavaScript*, le o campo de ordenación activo no formulario e inverte a dirección se coincide co botón pulsado, ou establece a orde ascendente por defecto en caso contrario.

Os criterios de ordenación definidos para cada vista son os seguintes:

- **Vista de programas:** título, campo de estudo, nivel EQF, créditos e país.
- **Vista de compoñentes educativos:** título, programa pai, créditos, cuatrimestre, campo de estudo e país.
- **Catálogo de institución, programas:** título, campo de estudo, nivel EQF e créditos.
- **Catálogo de institución, compoñentes educativos:** título, programa pai, campo de estudo, créditos e cuatrimestre.

Os criterios de tipo textual (título, campo de estudo, programa pai e país) ordénanse de forma alfabética; os de tipo numérico (créditos, nivel EQF e cuatrimestre) ordénanse por valor numérico.

Algúns destes criterios implican campos que non pertencen directamente ao nodo listado, senón a entidades relacionadas. O país reside no nodo institución referenciado polo programa; o campo de estudo e o título do programa pai, tanto na vista de compoñentes educativos como na páxina de compoñentes educativos do catálogo de institución, pertencen ao nodo programa referenciado polo IEC. Nestes casos, a ordenación resólvese a través das relacións entre entidades xa declaradas na configuración da vista, sen necesidade de definir unións adicionais entre táboas.

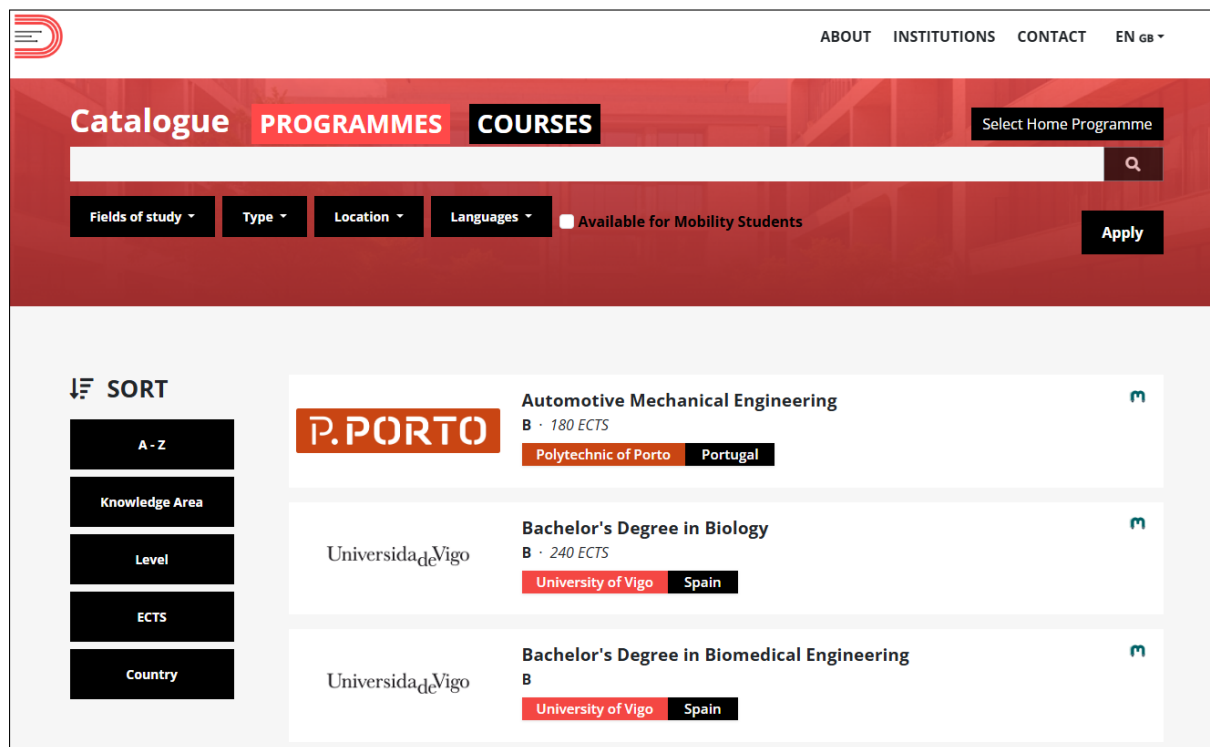


Figura 2: Botóns de ordenación dispoñibles na vista de programas.

3.2. Busca difusa

A busca difusa articulouse en tres compoñentes, aplicados de forma idéntica ás vistas de busca e á vista do catálogo de institución:

- **Índices de Search API:** un índice para cada tipo de contido (programas e compoñentes educativos), que proporcionan os campos sobre os que opera a clase de puntuación: título e nome da institución para os programas, e título, nome da institución e título do programa asociado para os compoñentes educativos (véxase o anexo A.1).
- **Hooks de Views:** dous puntos de extensión de Drupal que integran a lóxica de busca difusa na execución das vistas.
- **Clase PHP de puntuación:** calcula a similitude entre a consulta e o conxunto de campos indexados de cada candidato mediante a distancia de Levenshtein (véxase o anexo A.2).

Fronte a unha consulta SQL directa ás táboas de nodos, os índices ofrecen a vantaxe de desacoplar a lóxica de puntuación do esquema da base de datos e de manterse actualizados de forma automática ao modificarse o contido. Ambos os índices están configurados para cubrir todos os idiomas dispoñibles, de xeito que a busca non queda restrinxida ao idioma da interface.

A decisión de interceptar a consulta mediante *hooks*, en lugar de actuar sobre a capa de presentación, responde ao obxectivo de que o mecanismo sexa transparente para o usuario e compatible co sistema *AJAX* de *Views*.

A puntuación aplícase a nivel de palabra, e non sobre a frase completa, porque nun título formado por varias palabras un único erro tipográfico impediría calquera coincidencia ao comparar as cadeas enteiras. Comparando palabra a palabra, cada termo avalíase de forma independente e o resultado global reflicte mellor o grao real de similitude. Tanto o termo de busca como o texto indexado de cada candidato son normalizados previamente mediante conversión a minúsculas e eliminación de acentos, o que garante que a busca sexa insensible a maiúsculas e acentos, conforme ao requisito funcional establecido na sección 2. A continuación, tokenízanse en palabras de catro ou máis caracteres; o limiar de lonxitude mínima descarta preposicións, artigos e partículas curtas que, pola súa alta frecuencia, xerarían falsos positivos. A similitude calcúlase comparando cada palabra da consulta con cada palabra extraída do texto indexado do candidato, segundo os seguintes niveis:

- Coincidencia exacta: puntuación 1,0.
- Coincidencia de prefixo (a palabra da consulta é inicio dunha palabra do texto indexado): puntuación 0,85.
- Distancia de Levenshtein para palabras curtas (ata 5 caracteres): ata 1 erro permitido.
- Distancia de Levenshtein para palabras longas (máis de 5 caracteres): ata 3 erros permitidos.
- Distancia superior ao limiar: puntuación 0,0 (descartado).

A puntuación total dun elemento é a suma das mellores coincidencias obtidas para cada palabra da consulta. Esta escala continua permite ordenar os resultados por relevancia, a diferenza dun enfoque binario que simplemente incluíría ou excluíría resultados sen distinguir grao de axuste; en caso de empate na puntuación, aplícase como criterio de desempate a orde alfabética polo título. Os valores concretos de cada nivel e os limiares de distancia foron determinados de forma empírica a través dun proceso de calibrado (véxase o anexo B).

O fluxo de execución é o seguinte: cando o usuario introduce un termo, o *hook* de modificación de consulta intercepta a *query* SQL e invoca a clase de puntuación, que obtén os termos de busca a través do servizo de tradución automática descrito na sección 3.3, carga todos os candidatos indexados en *Search API*, normaliza e tokeniza tanto os termos de busca como o texto de cada candidato, e calcula a puntuación de forma independente para

cada termo; os resultados combínanse conservando a puntuación máxima por candidato. Con esa información, elimina a condición de filtrado por subcadea exacta, substitúea por un filtro polo conxunto de identificadores dos elementos que superaron o limiar de puntuación e limpa os criterios de ordenación preexistentes da vista. A orde de relevancia dos candidatos trasládase ao *hook* de pre-execución mediante un mecanismo de almacenamento compartido; este, xusto antes de que *Views* execute a consulta, incorpora á SQL un criterio de ordenación que garante que os resultados se devolven directamente na orde de relevancia correcta, paxinación incluída (como se ilustra na figura 3).

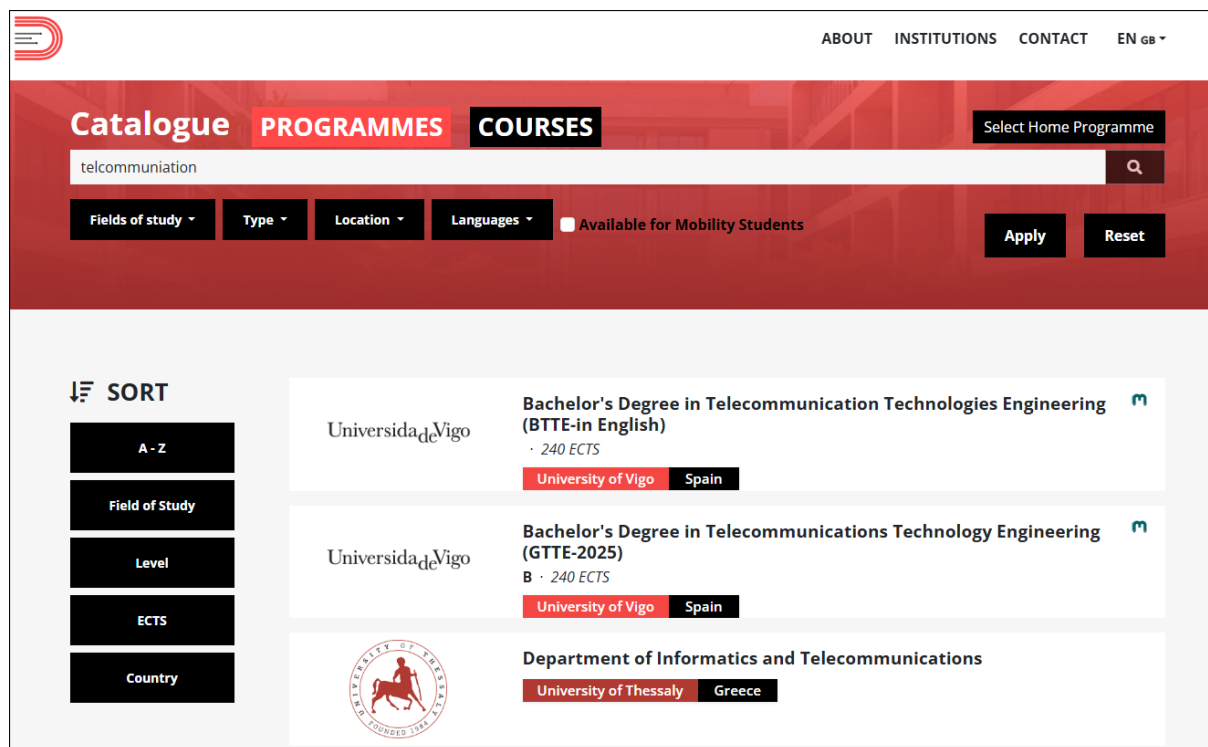


Figura 3: Resultados da busca difusa para a consulta *telcommunication*, con erros tipográficos respecto a *Telecommunications*.

3.3. Tradución automática da consulta

O catálogo de DACEM almacena cada contido obrigatoriamente en inglés, mentres que as traducións a outros idiomas son opcionais. Un usuario que formule a consulta noutro idioma pode non obter resultados pertinentes para contidos que carezan de tradución ao seu idioma, xa que o mecanismo de busca difusa compara o termo introducido directamente contra o texto indexado, sen ningún proceso de equivalencia semántica entre idiomas. Para cubrir esta limitación, incorpórase un servizo de tradución automática que obtén o equivalente en inglés do termo de busca e o engade como termo adicional de consulta. A busca execútase entón dúas veces —co termo orixinal e co traducido—, comparando cada un contra o conxunto completo do índice con independencia do idioma de cada contido; os resultados combínanse conservando a puntuación máxima por candidato. A tradución realízase unicamente ao inglés, idioma sempre dispoñible no catálogo, polo que basta cunha única tradución por consulta para cubri-lo por completo. O servizo articulouse en dous compoñentes:

- **Diccionario estático de termos académicos:** cobre o vocabulario máis habitual do ámbito educativo, con entradas separadas para programas e para compoñentes educativos.
- **Motor de tradución automática neural:** empregado como alternativa cando o termo non está recollido no diccionario.

O idioma de orixe da consulta determínase a partir do idioma activo na interface de Drupal, en lugar de aplicar detección automática sobre o texto introducido. Esta decisión responde á escasa fiabilidade da detección automática de idioma do motor de tradución sobre termos académicos curtos e frecuentemente sen acentos: termos como *gestion empresarial* ou *matemáticas aplicadas* son detectados incorrectamente como francés ou portugués con alta confianza, debido á semellanza léxica entre os idiomas románicos. O idioma da interface, en cambio, é un dato fiable e sempre dispoñible.

O dicionario estático consultase en primeiro lugar por ser máis rápido e preciso que o motor de tradución neural para vocabulario académico: os motores de propósito xeral tenden a producir traducións incorrectas con palabras soltas e sen contexto (por exemplo, *informática* pode traducirse como *it* en lugar de *computer science*). O dicionario mantén entradas separadas para termos de programas e para termos de compoñentes educativos, de xeito que cada chamada ao servizo consulta unicamente o subconxunto correspondente ao tipo de busca en curso, evitando percorrer entradas irrelevantes.

O termo orixinal da consulta inclúese sempre no conxunto de termos de busca, tanto se se obtén tradución como se non. Deste xeito cóbrense tres escenarios: se o contido dispón de tradución ao idioma do usuario, o termo orixinal pode puntuar directamente contra ela, ofrecendo potencialmente maior precisión que a comparación mediante o equivalente en inglés; se o usuario escribe directamente en inglés, a busca funciona sen necesidade de ningún proceso de tradución; e se a tradución falla ou produce un resultado de baixa calidade, o mecanismo de busca difusa opera igualmente sobre o termo orixinal como salvagarda.

O fluxo de execución é o seguinte: cando o usuario introduce un termo, o servizo de tradución detecta o idioma da interface; se non é inglés, consulta primeiro o dicionario e, se o termo non está recollido, invoca o motor de tradución automática. O resultado é un conxunto de un ou dous termos —o orixinal e, se procede, a súa tradución ao inglés— que se pasan ao mecanismo de busca difusa. A puntuación calcúlase de forma independente para cada termo e consérvase a puntuación máxima obtida para cada candidato.

4. Desenvolvemento

4.1. Ordenación de resultados

A implementación da ordenación articulouse en tres capas coordinadas:

- A configuración declarativa das vistas.
- As plantillas Twig.
- A lóxica JavaScript.

Na configuración YAML de cada vista, cada criterio de ordenación declárase como exposto e asináuselle un identificador único (*field identifier*). Este identificador establece o contrato entre a configuración da vista e a capa JavaScript: o valor do atributo `data-sort-by` de cada botón no HTML debe coincidir exactamente con este identificador para que o mecanismo funcione correctamente.

Algúns criterios implican campos de entidades relacionadas; nestes casos, o criterio declara o parámetro de relación correspondente, reutilizando as unións entre entidades xa definidas na configuración da vista, sen necesidade de declarar unións adicionais:

- **Vista de programas:** a ordenación por país emprega a relación coa institución referenciada polo programa.
- **Vista de compoñentes educativos:** a ordenación por campo de estudo e por título do programa pai usa a relación co nodo programa referenciado polo IEC; a ordenación por país encadéase ademais a través da relación co nodo institución.

- **Catálogo de institución, compoñentes educativos:** a ordenación por campo de estudo e por título do programa pai usa a relación co nodo programa referenciado polo IEC.

Nas plantillas Twig de cada vista engadíronse os botóns de ordenación, cada un coa clase `js-sort` e o atributo `data-sort-by` co identificador do criterio correspondente. As plantillas de todas as vistas cargan a mesma biblioteca JavaScript.

A plataforma de partida xa incluía un script JavaScript con lóxica de detección de clics sobre os botóns `.js-sort` e o envío do formulario. Modificouse para implementar o comportamento de alternancia descrito na sección 3.1: en lugar de usar sempre a dirección especificada no atributo do botón, o script le agora o estado actual dos campos `sort_by` e `sort_order` do formulario e inverte a dirección se o mesmo criterio xa está activo, ou establece a dirección ascendente por defecto en caso contrario. Así mesmo, actualizouse o método de envío: en lugar de disparar un evento *change* sobre os campos do formulario, o script simula agora un clic directo no botón de envío, o que desencadea o mecanismo de *Views* e actualiza os resultados mediante AJAX sen recargar a páxina.

4.2. Busca difusa

A implementación da busca difusa concretouse en tres elementos:

- A configuración do servidor e dos índices de *Search API*.
- A extensión do módulo `custom_views_filters` mediante dous *hooks* de *Views*.
- A nova clase `FuzzySearch` que encapsula a lóxica de puntuación.

Creouse un servidor de *Search API* con *backend* de base de datos, configurado cun limiar de lonxitude mínima de palabra de 4 caracteres, coherente co mecanismo de tokenización da clase de puntuación; a indexación de frases e a concordancia parcial desactiváronse, xa que o índice se emprega unicamente como catálogo de campos sobre o que operar a métrica de Levenshtein, sen usar a busca nativa do módulo. Sobre este servidor configuráronse dous índices sen restrición de idioma: `programmes`, que indexa os nodos de tipo *programme* polos campos `title` e `institution` (título da institución referenciada polo programa); e `courses`, que indexa os nodos de tipo *course* polos campos `title`, `institution` (título da institución, accedida a través do programa asociado) e `programme` (título do programa ao que pertence o compoñente).

O *hook* `hook_views_query_alter` invócase antes de que *Views* execute a consulta, permitindo interceptala e modificala. Este *hook* xa existía no módulo `custom_views_filters` para outras vistas; engadiuse nel a lóxica de busca difusa para as vistas de busca e para a vista do catálogo de institución. Para cada vista, obtense o valor do campo `combine` do formulario exposto e invócase o método estático `FuzzySearch::scoreFromIndex()` co índice correspondente; este método encárgase internamente de obter os termos de busca a través do servizo de tradución, cargar os candidatos do índice, puntualos de forma independente para cada termo e combinar os resultados conservando a puntuación máxima por NID. A condición SQL orixinal do filtro `combine` identifícase polo operador `formula` e a presenza do termo de busca nos seus argumentos, e elimínase da *query*. En función do resultado: se a clase devolve `NULL` (o termo non produce tokens), non se aplica ningún filtro e móstranse todos os resultados; se devolve un array baleiro, engádese a condición `nid IN (0)` para forzar un resultado baleiro; se hai coincidencias, limpanse os criterios de ordenación da vista (`$query->orderby = []`), gárdanse os NIDs ordenados por relevancia na propiedade estática `$orderedNids` e engádese a condición `nid IN ({NIDs})`.

Engadiuse ademais o novo *hook* `hook_views_pre_execute`, que se invoca xusto antes de que *Views* execute a consulta. Este comproba se a propiedade `$orderedNids` non está baleira e, de ser así, obtén o obxecto `SelectInterface` da *query*, constrúe mediante `addExpression()` a expresión `FIELD(node_field_data.nid, n1, n2, ...)` cos NIDs ordenados e engádea como criterio `ORDER BY`. Deste xeito, a base de datos devolve as filas directamente na orde de relevancia, sen ningunha reordenación posterior en PHP, e a paxinación tamén a respecta.

A clase `FuzzySearch`, creada dentro do módulo `custom_views_filters`, centraliza a lóxica de puntuación. O punto de entrada é o método estático `scoreFromIndex()`, que recibe o texto de busca e o identificador do índice e devolve os candidatos ordenados por puntuación descendente, ou `NULL` se o texto non produce ningún token válido. Este delega en dous métodos: `loadCandidatesFromIndex()`, que obtén os candidatos do índice, e `scoreAndFilter()`, que aplica a puntuación. A clase mantén ademais a propiedade estática `$orderedNids`, que actúa como canle de comunicación entre os dous *hooks*: `hook_views_query_alter` escribela cos NIDs ordenados por puntuación e `hook_views_pre_execute` léela para construír a expresión `FIELD()`.

`loadCandidatesFromIndex()` consulta o índice de *Search API* mediante unha *query* sen restrición de idioma e cun rango de ata 10.000 resultados. O identificador de cada ítem ten o formato `entity:node/{nid}:{lang}`; o NID extráese deste identificador mediante expresión regular. Para cada ítem, itera sobre os campos indexados e concatena os seus valores nun único texto, que é o que se pasa á fase de puntuación.

`scoreAndFilter()` normaliza e tokeniza o termo de busca e, para cada candidato, compara cada palabra da consulta con cada palabra dos campos indexados do candidato, acumulando a mellor puntuación obtida. Dado que *Search API* indexa cada tradución dun nodo como un ítem independente, o mesmo NID pode aparecer varias veces; por iso, os resultados indexanse por NID e, en caso de duplicado, consérvase a puntuación máis alta. Finalmente, os resultados ordénanse por puntuación descendente; en caso de empate, aplícase como criterio de desempate a orde alfabética polo título do candidato.

A tokenización divide o texto en palabras de 4 ou máis caracteres mediante a expresión regular `[a-z]{4,}`. O método `wordScore()` compara dúas palabras normalizadas e devolve unha puntuación entre 0 e 1. Os primeiros casos son a coincidencia exacta, que retorna unha puntuación de 1, e a coincidencia de prefixo, que retorna unha puntuación de 0,85. Para os demais, calcúlase a distancia de Levenshtein mediante a función `levenshtein()` de PHP e compárase cun limiar (isto é, a cantidade máxima de erros permitidos): se o supera, a puntuación é 0; se non, aplícase a fórmula $1 - d/L$, onde d é a distancia e L a lonxitude da maior palabra das dúas, de xeito que máis erros implican menor puntuación. O limiar é 1 para palabras de ata 5 caracteres e 3 para as máis longas.

4.3. Tradución automática da consulta

A implementación do servizo de tradución articulouse en tres elementos:

- A configuración do motor *LibreTranslate* [6] no entorno de desenvolvemento (véxase o anexo A.3).
- A clase `TranslationDictionary`, que almacena o vocabulario académico estático.
- A clase `TranslationService`, que coordina o dicionario e o motor e serve de punto de entrada para a clase `FuzzySearch`.

O motor de tradución automática neural *LibreTranslate* integróuse na contorna de desenvolvemento DDEV como un servizo Docker adicional, accesible desde o módulo PHP na URL interna `http://libretranslate:5000`. O límite de memoria do contedor estableceuse en 1GB para evitar que o proceso comprometa os recursos da máquina en contornas de desenvolvemento. A comunicación co servizo realízase mediante peticións POST ao punto de entrada `/translate`, que reciben o texto de entrada, o idioma de orixe e o idioma destino, e devolven o texto traducido en formato JSON; un *timeout* de 5 segundos garante que un fallo de comunicación non bloquee a busca, retornando `NULL` en caso de excepción.

A clase `TranslationDictionary` é unha clase PHP estática con dúas constantes: `PROGRAMMES`, con termos de titulacións, e `COURSES`, con termos de compoñentes educativos. Cada constante é un array indexado polo termo en inglés e cuxos valores son arrays de traducións por código de idioma, cubrindo os idiomas da plataforma (español, francés, checo, húngaro, esloveno, estoniano, galego, portugués e grego).

A clase `TranslationService` centraliza a lóxica de tradución. O punto de entrada é o método estático `getSearchTerms()`, que recibe o texto de busca e o identificador do índice, e devolve

un array de un ou dous termos. O idioma de orixe obtense mediante `uiLanguage()`, que recupera o idioma activo da interface de Drupal; se o idioma é inglés, `getSearchTerms()` retorna directamente o termo orixinal sen consultar ningún servizo. En caso contrario, consulta primeiro o dicionario correspondente ao tipo de índice e, se o termo non está recollido, invoca o motor de tradución. Se se obtén un resultado distinto do termo orixinal, engádese como segundo elemento do array.

`getSearchTerms()` delega a consulta ao dicionario en `dictionaryLookup()`, que á súa vez invoca `buildInvertedIndex()`. Para realizar a consulta de forma eficiente, este método transforma a estrutura do dicionario —indexada polo termo en inglés— nun índice invertido indexado por idioma e por termo normalizado, de xeito que a busca dun termo concreto nun idioma concreto é directa, sen percorrer todas as entradas. O índice constrúese a partir de `COURSES` para buscas de compoñentes educativos e de `PROGRAMMES` para buscas de programas. A comparación realízase sobre texto normalizado: `normalize()` converte a minúsculas, elimina diacríticos mediante transliteración Unicode e colapsa espazos, aplicándose tanto ao termo de entrada como ás claves do índice.

5. Conclusións e liñas futuras

(pendente)

6. Referencias

- [1] J. Murel and M. Syed, “What is information retrieval?” <https://www.ibm.com/think/topics/information-retrieval>, consultado: abril de 2026.
- [2] “Retrieval accuracy,” <https://www.sciencedirect.com/topics/computer-science/retrieval-accuracy>, consultado: abril de 2026.
- [3] “Digitising academic catalogues for enhanced mobility,” <https://projects.uni-foundation.eu/dacem/>, European University Foundation, 07/2024, consultado: abril de 2026.
- [4] W3Techs, “Usage statistics and market share of Drupal,” <https://w3techs.com/technologies/details/cm-drupal>, consultado: xuño de 2026.
- [5] M. Keran *et al.*, “Better Exposed Filters,” https://www.drupal.org/project/better_exposed_filters, consultado: xuño de 2026.
- [6] LibreTranslate, “LibreTranslate: Free and open source machine translation API,” <https://libretranslate.com/>, consultado: xuño de 2026.
- [7] T. Seidl *et al.*, “Search API,” https://www.drupal.org/project/search_api, 2024, consultado: maio de 2026.
- [8] Apache Software Foundation, “Apache Solr,” <https://solr.apache.org>, consultado: maio de 2026.
- [9] Elastic, “Elasticsearch,” <https://www.elastic.co/elasticsearch>, consultado: maio de 2026.
- [10] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.
- [11] F. J. Damerau, “A technique for computer detection and correction of spelling errors,” *Communications of the ACM*, vol. 7, no. 3, pp. 171–176, 1964.
- [12] W. E. Winkler, “String comparator metrics and enhanced decision rules in the Fellegi-Sunter model of record linkage,” in *Proceedings of the Section on Survey Research Methods*. American Statistical Association, 1990, pp. 354–359.
- [13] E. Ukkonen, “Approximate string-matching with q-grams and maximal matches,” *Theoretical Computer Science*, vol. 92, no. 1, pp. 191–211, 1992.
- [14] DeepL SE, “DeepL API,” <https://www.deepl.com/en/pro#api>, consultado: xuño de 2026.
- [15] Google LLC, “Cloud translation,” <https://cloud.google.com/translate>, consultado: xuño de 2026.
- [16] LibreTranslate, “Libretranslate API portal,” <https://portal.libretranslate.com/>, consultado: xuño de 2026.

7. Anexos

A. Estado da arte

A.1. Busca e indexación en sistemas de xestión de contidos

Os sistemas de xestión de contidos modernos incorporan habitualmente mecanismos de busca que van máis alá da simple consulta SQL. En Drupal 10, a solución integrada por defecto para a busca en listaxes de contido é o módulo *Views*, cuxo filtro *combine* realiza comparacións de subcadeas mediante operadores SQL LIKE. Esta aproximación limítase a coincidencias exactas e non ofrece capacidades de indexación nin de ponderación de resultados por relevancia.

Para cubrir estas limitacións, o ecosistema de Drupal conta co módulo *Search API* [7], que proporciona unha capa de abstracción para a indexación e consulta de contido. O módulo inclúe un sistema de procesadores configurables (como normalización de maiúsculas, transliteración de acentos, tokenización ou eliminación de palabras baleiras) que se aplican durante a indexación con independencia do *backend* escollido. Os *backends* dispoñibles máis empregados son os seguintes:

- **Backend de base de datos:** almacena o índice en táboas da base de datos relacional existente. Non require infraestrutura adicional e ofrece un rendemento axeitado para catálogos de tamaño moderado.
- **Apache Solr** [8]: motor de busca de código aberto baseado en Lucene, optimizado para grandes volumes de datos e con capacidades avanzadas de ponderación de resultados por relevancia. Require un servidor Solr externo.
- **Elasticsearch** [9]: solución distribuída orientada á alta dispoñibilidade e á escalabilidade horizontal, con capacidades similares a Solr. Tamén require infraestrutura adicional dedicada.

Para este traballo optouse polo *backend* de base de datos, dado que non require infraestrutura adicional, é suficiente para o volume de datos do catálogo DACEM, e permite integrar a lóxica de busca difusa directamente en PHP, empregando o índice para recuperar os títulos sobre os que aplicar a métrica de Levenshtein.

A.2. Busca difusa e correspondencia aproximada de cadeas

A busca difusa (*fuzzy search*) é unha técnica de recuperación de información que permite localizar resultados relevantes aínda cando a consulta do usuario non coincide exactamente co texto almacenado. A diferenza da busca por subcadeas exactas, tolera desviacións ortográficas e erros tipográficos, o que mellora a experiencia de usuario en sistemas con datos introducidos manualmente ou con vocabulario técnico susceptible a equivocacións.

O fundamento teórico da busca difusa reside no concepto de distancia de edición entre dúas cadeas, definida como o número mínimo de operacións elementais necesarias para transformar unha cadea na outra. O algoritmo máis empregado é a **distancia de Levenshtein** [10], que considera tres operacións básicas: inserción, eliminación e substitución dun carácter. Dados dous textos de lonxitude m e n , calcúlase en tempo $O(m \times n)$ mediante programación dinámica.

A **distancia de Damerau-Levenshtein** [11] amplía este modelo incorporando as transposicións de caracteres adxacentes como operación atómica, o que a fai máis precisa para modelar erros tipográficos humanos, onde o intercambio de dúas letras consecutivas é un dos erros máis frecuentes. Outra métrica habitual é a **similitude de Jaro-Winkler** [12], que premia as cadeas con prefixo común e adoita empregarse na comparación de nomes propios. Por último, os enfoques baseados en **n-gramas** [13] dividen as cadeas en secuencias solapadas de n caracteres e comparan a proporción de n-gramas compartidos, ofrecendo bo rendemento en textos longos e sendo facilmente paralelizables.

Para o presente traballo optouse pola distancia de Levenshtein como criterio de similaridade, pola súa sinxeleza conceptual, a súa robustez ante os tipos de erro tipográfico máis

comúns e a súa idoneidade para cadeas curtas como os títulos de programas e compoñentes educativos.

A.3. Selección do motor de tradución automática

Para a integración dun servizo de tradución automática nunha plataforma web, a opción máis habitual é o uso de APIs externas de tradución, que permiten enviar texto e recibir a tradución sen necesidade de despregar modelos propios. As principais alternativas consideradas foron as seguintes:

- **DeepL** [14]: API de tradución de pago considerada de alta calidade. Ao ser un servizo externo, as consultas dos usuarios abandonan o servidor da plataforma.
- **Google Cloud Translation** [15]: dispón dun nivel gratuíto de 500.000 caracteres mensuais, con tarifas de pago para volumes maiores. Comparte con DeepL a problemática de dependencia externa e saída de datos do servidor.
- **LibreTranslate** [16]: API de pago sen nivel gratuíto, con límites de petición por minuto. Comparte a problemática de dependencia externa das opcións anteriores.
- **LibreTranslate (autoaloxado)**: motor de código aberto despregable como contedor Docker, sen coste por petición, sen límites de cota e sen saída de datos do servidor.

Optouse pola opción autoaloxada de *LibreTranslate* polas seguintes razóns: elimina a dependencia de servizos externos de pago, garante que os datos das consultas permanecen no servidor da plataforma, non impón límites de petición e intégrase de forma natural no entorno de desenvolvemento DDEV mediante Docker.

O principal inconveniente de *LibreTranslate* é a calidade de tradución inferior á de alternativas comerciais, especialmente para termos curtos e illados sen contexto, como os habituais en nomes de programas e compoñentes educativos, que adoitan ser palabras soltas ou frases técnicas curtas. Ademais, o contedor require dunha cantidade non despreziable de RAM e presenta latencia nas primeiras peticións tras o arranque.

Estas limitacións mitigáronse con dúas medidas: a incorporación dun dicionario estático de termos académicos habituais, que cobre os casos de tradución incorrecta máis frecuentes e actúa como primeira opción antes de invocar o motor; e o establecemento dun límite de memoria de 1 GB no contedor, que acota o seu consumo de recursos.

B. Calibrado do algoritmo de busca difusa

O proceso de calibrado ten como obxectivo determinar os valores óptimos dos parámetros que controlan o comportamento do algoritmo: o limiar de distancia de edición, que establece o número máximo de erros tolerados para que dúas palabras se consideren similares, e a puntuación asignada á coincidencia de prefixo. Para iso, definíronse un conxunto de datos de proba e unha batería de consultas representativa dos escenarios de uso esperados. Avaliáronse cinco configuracións distintas e seleccionouse a que mellores resultados produce.

B.1. Datos de proba

Creáronse tres nodos de tipo *programme* con fins exclusivos de proba:

- *Engineering and Computer Science*
- *Medicine and Health Sciences*
- *Biotechnology*

B.2. Batería de consultas

Definíronse 11 consultas que cobren os escenarios de uso máis representativos, como se detalla na táboa 1.

Consulta	Resultado esperado	Escenario
engineering	aparece	Coincidencia exacta
enginering	aparece	1 erro tipográfico
enginiring	aparece	2 erros tipográficos en palabra longa
biotenology	aparece	2 erros tipográficos en palabra longa
helath	aparece	2 erros tipográficos en palabra de 6 caracteres
biotenolohy	aparece	3 erros tipográficos en palabra longa
engi	aparece	Coincidencia de prefixo
computer science	aparece	Consulta de varias palabras
engnrng	non aparece	Palabra excesivamente deformada
zzzz	non aparece	Termo irrelevante
eng	sen filtro	Non supera a lonxitude mínima

Táboa 1: Batería de consultas empregada no calibrado.

B.3. Parámetros e configuracións avaliadas

Avaliáronse cinco configuracións que varían o limiar de distancia de edición, a puntuación de prefixo e outros mecanismos de filtrado, co obxectivo de identificar a combinación de parámetros máis equilibrada. O limiar para palabras de ata 5 caracteres mantívose en 1 en todas as configuracións, dado que permitir 2 erros nunha palabra curta xeraría demasiados falsos positivos; o parámetro variable é o limiar para palabras de máis de 5 caracteres. As configuracións son:

- **Base:** limiar 2 para palabras longas, puntuación de prefixo 0,85. É a configuración inicial da que parten as demais.
- **Estrita:** limiar 1 para palabras longas e filtro adicional de puntuación mínima de 0,5 que descarta coincidencias débiles.
- **Laxa:** limiar 3 para palabras longas, o que permite ata 3 erros en palabras suficientemente extensas.
- **Sen prefixo:** idéntica á base, pero coa coincidencia de prefixo desactivada.
- **Limiar granular:** distingue tres rangos, con limiar 0 para palabras de ata 4 caracteres, 1 para palabras de 5 a 7 e 2 para as máis longas.

B.4. Resultados

A táboa 2 resume os resultados de cada configuración sobre a batería de consultas. A marca ✓ indica que o resultado obtido coincide co esperado; × indica que non.

Consulta	Base	Estrita	Laxa	Sen prefixo	Granular
engineering	✓	✓	✓	✓	✓
enginering	✓	✓	✓	✓	✓
enginiring	✓	×	✓	✓	✓
biotenology	✓	×	✓	✓	✓
helath	✓	×	✓	✓	×
biotenolohy	×	×	✓	×	×
engi	✓	✓	✓	×	✓
computer science	✓	✓	✓	✓	✓
engnrng	✓	✓	✓	✓	✓
zzzz	✓	✓	✓	✓	✓
eng	✓	✓	✓	✓	✓
Fallos	1	4	0	2	2

Táboa 2: Resultados das configuracións sobre a batería de consultas.

A configuración estrita rexeita demasiados casos válidos: o limiar reducido e o filtro de puntuación mínima descartan erros de dúas edicións que son perfectamente razoables. A

configuración sen prefixo falla en consultas polo inicio dunha palabra, o que supón unha perda de utilidade relevante para o usuario. A configuración granular, ao restrinxir o limiar a 1 edición para palabras de entre 5 e 7 caracteres, non tolera os 2 erros de `helath` (6 caracteres). A configuración base produce un único fallo en palabras longas con 3 erros tipográficos. A configuración laxa produce 0 fallos en toda a batería. Aumentar o limiar por encima de 3 ampliaría innecesariamente o risco de falsos positivos, polo que a configuración laxa representa o equilibrio máis axeitado entre tolerancia e precisión.

B.5. Configuración seleccionada

Seleccionouse a configuración laxa como resultado do proceso de calibrado. Os valores finais dos parámetros son os seguintes:

- **Puntuación de prefixo:** 0,85. Esta puntuación sitúase por riba das correspondencias por distancia de Levenshtein en palabras curtas (un erro nunha palabra de 4 caracteres dá $1 - 1/4 = 0,75$; nunha de 5 caracteres, $1 - 1/5 = 0,80$), e por debaixo da coincidencia exacta (1,0). Para palabras longas a relación invertece: 1 edición nunha palabra de 7 ou máis caracteres supera 0,85, o que reflicte que especificar case toda a palabra con un erro é máis preciso que quedarse a medias.
- **Limiar de distancia de edición:** 1 para palabras de ata 5 caracteres e 3 para palabras de máis de 5 caracteres.
- **Fórmula de puntuación por distancia de edición:** $1 - d/L$, onde d é a distancia de Levenshtein e L a lonxitude da palabra máis longa das dúas comparadas.